

Sistemas Operativos 2026



Clase 5 U2 Semáforos-Mutexes-Monitores

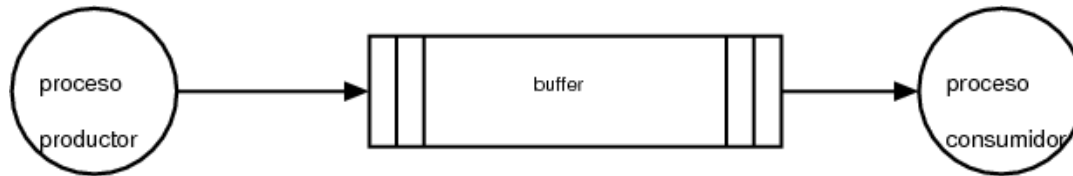
Ing. Juan Manuel Calvo

Nuestro problema

- Recursos (tales como la memoria) compartidos por más de un hilo.
- Las cosas se pueden complicar por las carreras, esto puede resultar en cálculos incorrectos y/o bloqueos.
- Por lo tanto necesitamos algo para mantener el orden
 - Exclusión mutua
 - Sincronización de hilos.
- Esto puede ser solucionado tanto en la aplicación o en el sistema operativo o en ambos.

Ejemplo: Problema del productor/consumidor

- Cada tanto el productor coloca un ítem en el buffer.
- Cada tanto el consumidor saca un ítem del buffer.
- se requiere una cuidadosa sincronización (ambos corren simultaneamente)
 - Si el buffer está vacío **el consumidor** debe esperar.
 - Si el buffer está lleno **el productor** debe esperar.
- La solución típica implica una variable compartida, la cuenta.
 - También conocido como el problema del buffer limitado.



Dormir y despertarse (sleep and wakeup)

- Dos llamadas al sistema abstractas (¡no son los nombres de llamadas al sistema reales!)
 - Sleep()
 - se bloquea hasta que alguien llama a wakeup.
 - wakeup(proceso)
 - desbloquea el proceso
 - el proceso se tiene que haber bloqueado previamente ejecutando un sleep().

Productor/Consumidor (solución parcial)

- Productor

```
while (TRUE) {  
    item = produce_item();  
    if (count == N)  
        sleep();  
    insert_item(item);  
    count = count + 1;  
    if (count == 1)  
        wakeup(consumer);  
}
```

- Consumidor

```
while (TRUE) {  
    if (count == 0)  
        sleep();  
    item = remove_item();  
    count = count - 1;  
    if (count == N - 1)  
        wakeup(producer);  
    consume_item(item);  
}
```

Productor/Consumidor (solución parcial)

- No funciona completamente
 - count es inicialmente 0
 - El consumidor lee la cuenta
 - Se produce una interrupción antes de ejecutar el sleep(), el control pasa del consumidor al productor.
 - El productor produce el item, lo inserta e incrementa la cuenta (a 1)
 - El productor ejecuta un wakeup, pero no hay un consumidor esperando en ese momento.
 - Ahora el consumidor continúa ejecutando y llama a sleep(), el consumidor se bloquea.
 - El consumidor queda bloqueado para siempre.
- Problema principal: hubo una carrera - el wakeup se perdió.

Semáforos

- Un semáforo S tiene un valor entero no negativo.
- Dos operaciones: down() y up()
 - down(s): si s es positivo, decrementa el valor de s, de lo contrario el proceso se duerme antes de decrementarlo.
 - up(s): si había algún proceso durmiendo lo despierta, de lo contrario incrementa s.
 - Usa una lista interna de procesos dormidos
 - Las llamadas up() y down() son acciones atómicas.
 - No hay espera activa

Uso de los Semáforos

El valor del semáforo S se inicializa en 1

```
Semaphore S = 1;
```

```
...
```

```
down(S)
```

```
región crítica
```

```
up(S)
```

```
...
```

Implementación de los semáforos

- Implementados en el kernel
- Enmascarar las interrupciones cuando se opera sobre el semáforo
- En multiprocesadores, usar también la protección de Test and Set
- Ambas duran solo unos pocos nanosegundos, no representan un problema

Implementación de los semáforos

```
typedef struct {
    int value;
    *pid_t wait_list; /* Lista de procesos */
} semaphore;

down(semaphore S) {
    if (S.value > 0) S.value--;
    else {
        add this process to S.wait_list;
        sleep;
    }
}

up(semaphore S){
    if (S.wait_list==null) S.value++;
    else {
        select and remove a process P from S.wait_list;
        wakeup(P);
    }
}
```

up() y down() tienen que ser protegidos como secciones críticas.

Problema del productor/consumidor, revisado



- Exclusión mutua para acceder al buffer
- ¿Qué sucede si cambiamos el orden de los down y up?

```
semaphore: full = 0 /* number of full slots */
           empty = SLOTS /* number of empty slots */
           mutex = 1 /* critical section (binary) */
```

Producer code:

```
while (TRUE) {
    /* produce */
    down (empty)
// -----
    down (mutex)
    /* add to buffer */
    up (mutex)
// -----
    up (full)
}
```

Consumer code:

```
while (TRUE) {
    down (full)
// -----
    down (mutex)
    /* remove from buffer */
    up (mutex)
// -----
    up (empty)
    /* consume */
}
```

Ver prod_cons_sem.c

Dos usos diferentes de los semáforos

- Semáforos binarios, mutex
 - Usados para obtener exclusión mutua
 - definir inicialmente $S=1$.
- Semáforos contadores
 - Usan valores iniciales de $S > 1$ para permitir varios procesos activos simultaneamente.
 - Tienen otros usos distintos a la exclusión mutua (Ej: control de carga de recursos, sincronización).

Mutex

- Versión simplificada de un semáforo
 - Sólo dos estados:
 - 0 significa desbloqueado
 - 1 significa bloqueado
- Se usan para garantizar la exclusión mutua
- Son fáciles y eficientes para implementar
 - si está disponible la instrucción TSL se pueden implementar en espacio de usuario

Mutex

- Ejemplo de implementación:

mutex_lock:

```
TSL REGISTER,MUTEX | copy mutex to register and set mutex to 1
CMP REGISTER,#0    | was mutex zero?
JZE ok             | if it was zero, mutex was unlocked, so return
CALL thread_yield  | mutex is busy; schedule another thread
JMP mutex_lock     | try again
```

ok:

```
RET                | return to caller; critical region entered
```

mutex_unlock:

```
MOVE MUTEX,#0      | store a 0 in mutex
RET                | return to caller
```



UNIVERSIDAD
DEL CEMA

UCEMA

Mutexes en pthreads

Llamada	Descripción
<code>pthread_mutex_init</code>	Crea un mutex
<code>pthread_mutex_destroy</code>	Destruye un mutex existente
<code>pthread_mutex_lock</code>	Obtener un lock o bloquearse
<code>pthread_mutex_trylock</code>	Obtener un lock o fallar
<code>pthread_mutex_unlock</code>	Liberar un lock

Uso de los semáforos

- Veamos que sucede si intercambiamos el orden de los down's en el código del productor.
- Si el buffer está lleno, el productor se bloqueará con el mutex en 0. Después, la próxima vez que el consumidor trate de acceder al buffer hará el down del mutex y también se bloqueará.
- Habremos llegado a un bloqueo mutuo.
- Debemos ser muy cuidadosos cuando usamos semáforos, un solo error y todo el sistema se detiene.

Los riesgos de los semáforos y los mutex

- Usar semáforos y mutex correctamente es difícil.
- Si se hace mal, se puede producir un bloqueo.
- Realizar pruebas es difícil porque todo depende de la secuencia de eventos.
- Necesitamos una construcción de alto nivel.

Monitores

- Inventados por Hoare (1974) y Brinch Hansen (1975)
- Son una construcción del lenguaje de programación
- Están en Java pero no C y C++.
- Un monitor es como una clase, pero solo se puede ejecutar un procedimiento a la vez.
- En otras palabras, son una implementación en el lenguaje de los mutex.

Ejemplo monitores

monitor example

```
integer i;  
condition c;
```

```
procedure producer( );  
    ...  
end;
```

```
procedure consumer( );  
    ...  
end;
```

```
end monitor;
```

[Ver ejemplo ProducerConsumer.java](#)

Monitores

- Los monitores tienen una importante propiedad que los hace útiles para exclusion mutua:
 - en un instante dado solamente un proceso puede estar activo en un monitor.

Monitores

- Los monitores son una construcción de los lenguajes.
 - el compilador los tiene que tratar especialmente.
 - cuando un proceso llama a un procedimiento del monitor el compilador debe agregar instrucciones para chequear que no haya otro proceso activo.
 - si hay uno activo debe suspenderse hasta que el otro proceso deje el monitor.
 - es un trabajo del compilador implementar la exclusión mutua, por ejemplo usando un mutex.
- dado que es el compilador el que arregla las cosas para la exclusión mutua, es mucho más difícil que algo no funcione.
 - el programador no necesita saber como el compilador se arregla para hacer la exclusión mutua.
 - solamente es suficiente conocer que colocando todas las regiones críticas en procedimientos de un monitor, no sucederá que dos procesos ejecuten sus regiones críticas al mismo tiempo.

Monitores - variables de condición

- Los monitores dan una manera simple de lograr la exclusión mutua, pero se necesita algo más:
 - Necesitamos que los procesos se bloqueen cuando no pueden continuar.
 - para esto son las variables de condición y las dos operaciones sobre ellas: wait y signal.
- cuando un procedimiento del monitor descubre que no puede continuar, hace un wait en una variable de condición, el procedimiento se bloquea y otro que no había podido entrar puede hacerlo.
 - ¿qué se hace después de sucede después del signal?
 - Hoare propuso que el proceso bloqueado corriera
 - Brinch Hansen propuso que el proceso que hace un signal debe salir del monitor inmediatamente.

Variables de condición

- Wait y Signal pueden parecer similares a sleep y wakeup que vimos antes los cuales sufrían de una carrera fatal.
- sleep y wakeup fallaron porque mientras un hilo está tratando de ir a sleep el otro está tratando de despertarlo, con monitores no puede suceder.
 - El procedimiento dentro del monitor puede completar su wait sin preocuparse que el planificador lo pueda cambiar por otro.
- Las variables de condición NO son condiciones
 - Así que nunca hagan:

```
if (conditionVariable) {  
    ...  
}
```
 - En cambio, es una forma de que un hilo espere (si algún recurso no está disponible) y otro hilo pueda activarlo una vez que el recurso esté disponible

Monitores - variables de condición

- si el signal se hace sobre una variable de condición que tiene varios procesos esperando, solo uno es revivido (determinado por el planificador del sistema).



Las variables de condición y los locks

- La variable de condición no reemplaza los locks, sino que los complementan
- `sleep(condición, bloqueo)` O `wait(condición, bloqueo)`
- Primero suelte el lock, coloque el hilo en la cola de la condición, si se despierta, recupere el bloqueo
- Una vez que el Sleep regresa, despertado por algún otro hilo y también mantiene el bloqueo
- `wake(condición)` O `signal(condición)`: despierta un hilo esperando la condición(cola)
 - Algunos sistemas usan un nombre diferente como `notify(condición)`

Variables de condición

- Los **mutexes** son buenos para permitir o bloquear el acceso a una **región crítica**.
- Las **variables de condición** permiten que los subprocesos se bloqueen debido a que **no se cumple alguna condición**.
 - Casi siempre los dos métodos se usan juntos.

Variables de condición

- `wait - signal`
- Las variables de condición no son contadores, no se acumulan para uso posterior como los semáforos.
 - si una variable de condición es señalada y nadie está esperando esta se pierde.
- el `wait` debe venir antes del `signal`.
 - esto facilita la implementación.
 - Las señales perdidas no son un problema.

Variables de condición

Llamada al sistema de hilos	Descripción
<code>pthread_cond_init</code>	Crear una variable de condición
<code>pthread_cond_destroy</code>	Destruir una variable de condición
<code>pthread_cond_wait</code>	Bloquearse esperando una señal
<code>pthread_cond_signal</code>	Señalizar otro hilo y despertarlo
<code>pthread_cond_broadcast</code>	Señalizar múltiples hilos y despertarlos a todos

Variables de condición

- Java usa wait y notify
- Muchos otros lenguajes usan el modelo de Java.



Desventajas de los monitores

- Los monitores son útiles, pero solo están disponibles en unos pocos lenguajes.
- Los monitores, semáforos y mutex son útiles dentro de una sola máquina, incluso en un multiprocesador.
 - No funcionan bien sin memoria compartida.
- ¿Qué ocurre con un sistema distribuido conectado a través de una LAN?

Paso de mensajes

- Modelo basado en estas dos operaciones de red:
 - `send(destino, &mensaje);`
 - `recv(origen, &mensaje);`
- El receptor se bloquea si no hay datos disponibles.
- El emisor se bloquea si los búferes del receptor están llenos.
- Cuestiones complejas de red, como la confirmación de recepción, el control de flujo, la detección y corrección de errores, la ordenación de mensajes y la autenticación.
 - Temas de otra materia.

Uso del paso de mensajes

- Muchos diferentes paradigmas de como usarlos
 - Rendezvous: un mensaje a la vez; el emisor y el receptor operan en sintonía
 - Buzones: búferes de tamaño fijo en el canal; el emisor los bloquea si están llenos
 - Solicitudes explícitas: mensajes vacíos del consumidor al productor

Productor consumidor con paso de mensajes

```
#define N 100                                /* number of slots in the buffer */
void producer(void)
{
    int item;

    message m;                                /* message buffer */

    while (TRUE) {
        item = produce_item( );              /* generate something to put in buffer
*/
        receive(consumer,&m);                 /* wait for an empty to arrive */
        build_message(&m, item);              /* construct a message to send */
        send(consumer,&m);                    /* send item to consumer */
    }
}
```

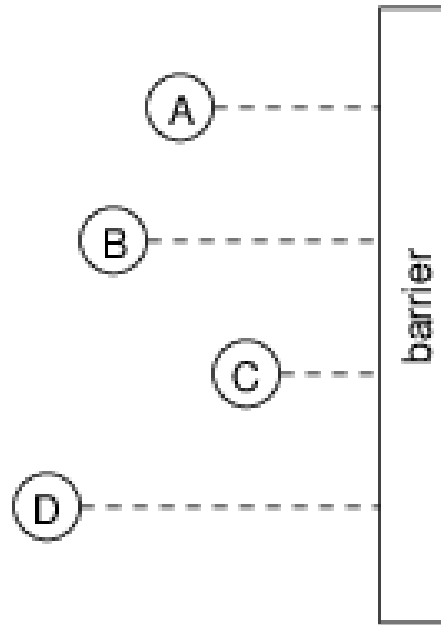
Productor consumidor con paso de mensajes



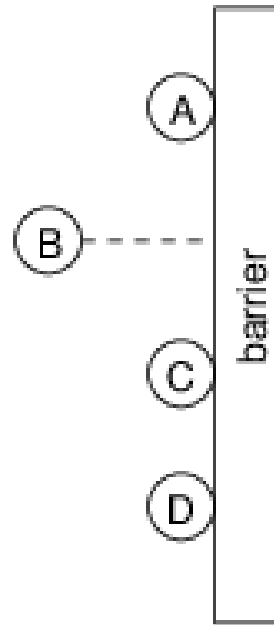
```
void consumer(void)
{
    int item, i;
    message m;
    for (i = 0; i < N; i++) send(producer, &m); /* send N empties */

    while (TRUE) {
        receive(producer, &m);          /* get message containing item
*/
        item = extract_item(&m);        /* extract item from message */
        send(producer, &m);              /* send back empty reply */
        consume_item(item);              /* do something with the item */
    }
}
```

Barreras



(a)



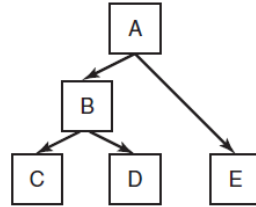
(b)



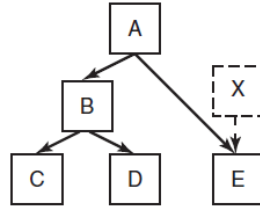
(c)

Evitando los locks: Read-Copy-Update

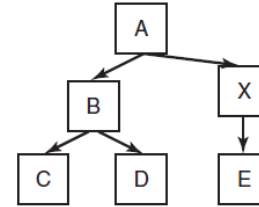
Adding a node:



(a) Original tree.

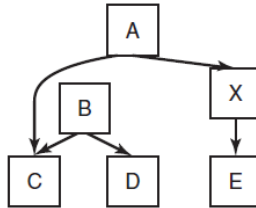


(b) Initialize node X and connect E to X. Any readers in A and E are not affected.

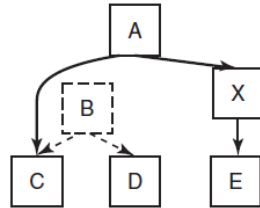


(c) When X is completely initialized, connect X to A. Readers currently in E will have read the old version, while readers in A will pick up the new version of the tree.

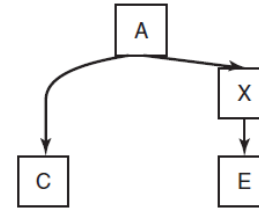
Removing nodes:



(d) Decouple B from A. Note that there may still be readers in B. All readers in B will see the old version of the tree, while all readers currently in A will see the new version.



(e) Wait until we are sure that all readers have left B and C. These nodes cannot be accessed any more.



(f) Now we can safely remove B and D

Resumen

- El IPC (interprocess communication) es complejo
- Existen muchos esquemas posibles, según el entorno y el sistema operativo subyacente
- Existe el peligro de bloqueos: tema para una próxima clase...

